XINGQIANG CHEN

Hangzhou, China
+86 150-0133-4125 ◇ joy6677@outlook.com ◇ github.com/chenxingqiang

OBJECTIVE

Looking for **FULL TIME** roles in **AI Accelerator Compiler / System Software** · **LLM Inference Systems** · **HW/SW Co-design**.

INTERDISCIPLINARY BACKGROUND

Physics and biophysics training across computational chemistry, molecular dynamics, and computer science. Contests: **CUMCM National First Prize** (2012); **MCM Meritorious Winner** (2013); **Second Prize**, CCF Open Source GPU Innovation Ecosystem Regular Season S1, GPU Operator Optimization Challenge (2025).

EDUCATION

Doctoral Studies in Biophysics and Condensed Matter Sep. 2014 - Jul. 2018
Xiamen University, Department of Physics and Engineering; protein dynamics and QM/MM simulation
Certificate of completion; degree defense in progress

B.S. in Theoretical Physics Sep. 2010 - Jul. 2014
Xiamen University, Department of Physics and Engineering

SKILLS AND PROJECTS

Languages: C++, Python (primary); CUDA, Java, Scala, Go.

Compiler / Toolchain: MLIR, LLVM backends and ISA bring-up, TVM, graph partitioning, operator fusion, cost modeling, codegen, auto-tuning.

Chip Software / Verification: Palladium emulation, GDB co-debug against simulation models, firmware and weight-map generation, operator-level metrics (Cycles/MACs).

Inference Systems: Transformers, KV-Cache, quantization, batching and scheduling, Triton kernels, FlashAttention, CUPTI/Perfetto profiling.

Platforms: PyTorch, TensorFlow, ONNX, Kubernetes; Spark, Flink, Hadoop.

Open Source & Releases (`github.com/chenxingqiang` unless noted):

- **TraceSmith** — GPU profiling & replay across CUDA/ROCm/Metal/MACA/Ascend; C++ core, Python bindings, on PyPI. `TraceSmith`
- **YiRage Persistent Kernel** — LLM megakernel compiler and runtime, multi-backend. `YiRage`
- **JAX-sklearn** — sklearn-compatible API on JAX; 13,058 tests passing; on PyPI. `jax-sklearn`
- **Secret-Learn** — Privacy-preserving ML: federated learning, secret sharing, split learning. `Secret-Learn`
- **YouTube-DNN-RecSys** — TensorFlow + Spark retrieval and ranking (60 stars). `YouTube-DNN-RecSys`
- **Research CLI** — Command-line AI tooling for research workflows. `iechor-research/research-cli`
- **Questions-Gen** — Math-problem generation (Qwen3-14B, GRPO, distillation); CLI and HF weights. `GenQuestion`

EXPERIENCE

Compiler & Toolchain Expert, Witmem Technology
Hangzhou, China
Compiler / Palladium Verification *Dec. 10, 2025 - Present*

- Owned the product-line compiler and Palladium emulation for a CIM chip: model compile, codegen, on-chip execution, and defect isolation on the emulator.
- Brought up software on Palladium: ran compiler output on the emulator and jointly debugged with GDB across the simulation model, silicon, and system software.

- Implemented **CIM-MLIR**: PyTorch/TensorFlow/ONNX lowered via TOSA/StableHLO to CPU, GPU, and CIM, covering frontend, graph partition, hardware backend, and runtime.
- Implemented cost-driven graph partition and CIMHigh/Mid/Low IR; used OR-Tools for Die/L2/L0 allocation; emitted burnable **config.bin**, weight **map.bin**, GPU PTX, and LLVM host code.
- Implemented a cross-device runtime: per-device executors, unified memory, compute/transfer overlap and triple buffering; C++/Python APIs with latency, utilization, and transfer-cost counters.
- Implemented **GRC-PIM**: drift-gated, GEMV-reusing residual calibration and soft-decision error correction for PIM; submitted to *npj Unconventional Computing* (see Publications).

Senior AI Compiler Engineer, Yizhu Tech Ltd.
Hangzhou, China
R&D Department *Sep. 2022 - Dec. 2025*

- Led **YiRage Persistent Kernel (YPK)**, compiling end-to-end LLM decode into a single megakernel; **1.2×–6.7×** end-to-end latency gain over a PyTorch eager baseline on Llama2-7B, single A100 (varies with batch and sequence length).
- Implemented fused operators (RMSNorm+Linear, GQA) and a persistent execution model, cutting kernel-launch and intermediate-tensor round trips; largest gains on the batch=1 decode path.
- Built the scheduling layer and integrated Triton and FlashInfer kernels; multi-GPU scaling validated up to **8** devices with NVSHMEM-backed tensors.
- Developed graph compilation and auto-tuning: kernel parameter search and KV-cache-friendly memory layout to reduce memory traffic.
- Filed **2** invention patents; delivered the stack into **2** datacenter customer PoCs.
- Owned the LLVM backend Target and complex-ISA support for the in-house SoC, driving backend standardization and the software ecosystem.
- Built model-conversion and operator-level evaluation: integrated TVM, ONNX, MLCommons, TensorFlow/PyTorch into **YZlite**, reporting Cycles/MACs and multi-chip benchmarks. `ReRAM-Labs/yzlite`

Privacy-preserving Algorithms Engineer, Trustbe
Hangzhou, China
IT Group *Jan. 2022 - Sep. 2022*

- Built a homomorphic-encryption federated learning platform for cross-institution modeling in finance, improving model quality over local-only baselines while keeping privacy constraints.
- Designed MPC protocols for joint training and inference of bank risk models, balancing protocol security against communication cost.
- Open-sourced **Secret-Learn**: sklearn-compatible API across federated learning, secret sharing, and split learning, JAX-accelerated and integrated with SecretFlow; on PyPI. `Secret-Learn`
- Filed **3** invention patents, all published: `CN114925786A`, `CN114997345A`, `CN115130681A`.

Staff Algorithms Engineer, BioMap
Beijing, China
Algorithm Engineering Group *Sep. 2021 - Dec. 2021*

- Built bioinformatics tooling and a unified data platform for large-scale sequence analysis, structure prediction, and functional annotation.
- Set up evaluation pipelines for protein structure prediction models (AlphaFold family) feeding downstream drug design.
- Two registered software copyrights (2021SR2004464, 2021SR1862397); open-sourced AlphaFold2 code walkthroughs (26 stars). `alphafold2-codecs`

Senior Algorithms Engineer, Yalla Group (NYSE: YALA)
Hangzhou, China
Data Center Engineering Group *Sep. 2020 - Sep. 2021*

- Rebuilt the ChatRooms recommender with multi-objective ranking; A/B tests showed significant gains in daily active time and 7-day retention (**42% → 61%**).
- Built Ludo risk control from real-time behavior features and ML models, materially reducing fraudulent refunds.
- Developed a card-game AI with deep RL for human-vs-bot play and matchmaking fill, improving off-peak match quality and retention.

Senior Algorithms Engineer, Gejia Network Tech — Hangzhou, China
Search & Recommendation Engineering Group — *Mar. 2019 - Sep. 2020*

- Designed multi-scenario ranking with collaborative filtering, deep ranking models, and a product knowledge graph; online experiments delivered double-digit relative CTR and CVR gains.
- Optimized the serving path with feature caching and candidate pruning, holding P99 latency under **50ms** through peak promotional traffic.
- Open-sourced YouTube-DNN-RecSys, a TensorFlow + Spark retrieval and ranking reference (60 stars). `chenxingqiang/YouTube-DNN-RecSys`

Machine Learning Engineer, Bianlifeng Tech — Beijing, China
Data Engineering Group — *Mar. 2017 - Mar. 2019*

- Built store-level sales forecasting across **50+** stores using gridded weather and spatiotemporal features, reducing stock-outs and dead stock while improving turnover.
- Linked customer identities to purchase history over tens of millions of events to power personalized recommendations.
- Shipped a product recognition model for **500+** SKUs used in fast checkout, cutting checkout and queue time.

RESEARCH & INTERNSHIPS

Molecular Dynamics Simulation of Proteins with QM/MM model — Xiamen university, China
(China National Funds) — *Sep. 2014 - Mar. 2017*

- Ran QM/MM protein dynamics simulations and conformational analysis; results published in Acta Cryst. D, PLoS ONE, and Phys. Rev. E (see Publications).

Liquid Biopsy ct-DNA Kit and DNA Methylation Analysis — Lepu Medical, China
(R&D Engineer Intern) — *Mar. 2016 - Mar. 2017*

- Analyzed roughly 10,000 public gene-chip samples (TCGA/GEO) to localize Septin9 methylation sites, an early colon-cancer marker, informing biomarker selection for a clinical diagnostic kit.

AWARDS & SERVICE

- **National First Prize**, China Undergraduate Mathematical Contest in Modeling (CUMCM), 2012.
- **Meritorious Winner**, Mathematical Contest in Modeling (MCM/COMAP), 2013.
- **6th** worldwide, International Solar Decathlon Competition, 2013.
- Ranks **57** and **32** of 1739 teams, IEEE ICDM Global Meteorological AI Challenge, 2018.
- **Second Prize**, 8th CCF Open Source Innovation Competition — Open Source GPU Innovation Ecosystem Regular Season S1, GPU Operator Optimization Challenge (CCF/ODTC), 2025. CCF announcement
- Judge, first screening, Best Practices of Applied Algorithms, 2021.

PUBLICATIONS

- Chen X. GRC-PIM: A Drift-Gated, GEMV-Reusing Residual Calibrator for Soft-Decision Error Correction in Processing-in-Memory. *npj Unconventional Computing* (submitted).
- Cai Y, Su M, Ahmad A, Hu X, Sang J, Kong L, Chen X, Wang C, Shuai J, Han A. Conformational dynamics of the essential sensor histidine kinase WalK. *Acta Crystallographica Section D: Structural Biology.* 2017 Oct 1;73(10):793-803.
- Xia H, Chen X, Chen X, Cheng J, Liu Y, Chen X, Zhang Q, Zou G. Chiral discrimination of amino acid enantiomers based on different interactions with Cu2+. *Sensors and Actuators B: Chemical.* 2018 Jan 1;254:44-51.
- Ahmad A, Cai Y, Chen X, Shuai J, Han A. Conformational dynamics of response regulator RegX3 from Mycobacterium tuberculosis. *PloS one.* 2015 Jul 22;10(7):e0133389.
- Chen Y, Qi H, Li X, Cai M, Chen X, Liu W, Shuai J. Suppressing effect of C a 2+ blips on puff amplitudes by inhibiting channels to prevent recovery. *Physical Review E.* 2016 Aug 15;94(2):022411.

Open Source Architecture Appendix

Chen Xingqiang · github.com/chenxingqiang · one project overview and architecture per page

Primary flow
Fallback / feedback
Dashed = architecture zone
Box = module + responsibility

CONTENTS · one architecture per page

TraceSmith
GPU profiling & replay

YiRage
RuntimeFusion compiler

JAX-sklearn
transparent JAX acceleration

Secret-Learn
privacy-preserving ML

YouTube-DNN
two-tower recall

Research CLI
agent workflow

Questions-Gen
three-stage training

READING GUIDE

Read top to bottom. Dashed containers are architecture zones; boxes are modules and responsibilities.

Solid blue arrows show the primary path; dashed purple arrows show fallback or feedback paths.

TraceSmith — 系统架构 · GPU Trace 端到端数据流

Application → bindings → capture / state / format → replay & export → GPU backends

Primary flow
Fallback / feedback
Dashed = architecture zone
Box = module + responsibility

PROJECT OVERVIEW TraceSmith is a cross-platform GPU profiling and deterministic replay system for AI compiler, framework, and driver engineers.
Its C++17 core captures low-overhead execution events, stores compact SBT traces, and exports timelines for analysis and debugging.

APPLICATION · 用户入口

- **C++ / Python Application** — 嵌入业务进程
- **CLI** — profile · record · view · export
- **External UI** — Perfetto · Chrome

BINDINGS · Python / C++ 边界

- **_tracesmith.so** — pybind11 native extension

CORE LIBRARY · C++17

- **Capture Module** — CUPTIProfiler · MCPTIProfiler; Metal · Ascend · ROCm · eBPF
- **Common + Format** — RingBuffer SPSC · StackCapture; SBTWriter / SBTReader
- **State Module** — GPUStateMachine; TimelineBuilder · PerfettoExporter
- **Replay Module** — ReplayEngine · StreamScheduler; DeterminismChecker · FrameCapture

CROSS-CUTTING · 集群与调试

replay / debug feedback

- **Cluster Module** — GPUTopology · TimeSync; NCCLTracker
- **GDB / Tracy** — RSPHandler · GPUDebugEngine; TracyExporter

GPU BACKENDS · 系统接口

- **NVIDIA** — CUDA · CUPTI · nsys
- **MetaX** — MACA · MCPTI · mcTracer
- **Apple / AMD** — Metal · xctrace · ROCm
- **Huawei / Linux** — CANN · msprof · eBPF

OUTPUTS · Trace 产物

- **trace.sbt** — compact binary
- **trace.json / .perfetto** — Perfetto timeline
- **graph.dot / ASCII** — analysis views

主链：GPU event → lock-free ring → SBT/state → replay 或 Perfetto；Cluster / GDB 为旁路能力。

YiRage (YPK) — 系统架构 · RuntimeFusion 数据流

Inference engine owns scheduling/KV; RF owns fusion identity; search and execute use the same backend

Primary flow
Fallback / feedback
Dashed = architecture zone
Box = module + responsibility

PROJECT OVERVIEW YiRage is a multi-backend LLM kernel compiler and runtime centered on graph superoptimization and RuntimeFusion. It searches backend-specific fusion plans offline, then injects verified FusionCapsules into vLLM or SGLang decode steps.

INFERENCE ENGINE · 调度与 KV 所有者

- **vLLM** — continuous batch · PagedAttention
- **SGLang** — RadixAttention · extend meta
- **Decode Metadata** — block table · seq lens · SM budget

RUNTIMEFUSION · 每个 decode step

- **RF.step(inputs, meta)** — select 0..N capsules
- **FusionCapsule** — MLP gate_up + down segment pipeline
- **Native Fallback** — skip → engine native ops

SEARCH & PLAN · offline / warm-up

- **superoptimize(backend)** — KN / TB graph search
- **FusionPlan / μGraph** — profile · fingerprint verify
- **Persistent Cache** — shape bucket restore
- **Ray Coordinator** — optional distributed search — fallback / re-search

BACKEND MANAGER · 编译与执行边界

- **BackendRegistry** — search + execute same backend
- **KNGraph / TBGraph** — transpiler → kernels
- **yirage.core** — Cython runtime

BACKENDS → HARDWARE

- **CUDA / MACA** — NVIDIA GPU · MetaX C500/C550
- **ROCm / MPS** — AMD GPU · Apple Silicon
- **Ascend / NKI** — Huawei 910 · AWS Neuron
- **CPU / MLIR / Triton** — x86 · ARM · extra targets

主链：Engine metadata → RF.step → FusionCapsule → BackendRegistry → target hardware。

JAX-sklearn (xlearn) — 五层架构 · 透明加速数据流

100% sklearn API → runtime injection → proxy dispatch → array conversion → JAX hardware

Primary flow
Fallback / feedback
Dashed = architecture zone
Box = module + responsibility

PROJECT OVERVIEW JAX-sklearn provides a drop-in scikit-learn API while transparently accelerating supported estimators with JAX.
Runtime proxies select JAX or native sklearn by algorithm, data size, and platform, preserving familiar inputs and outputs.

USER CODE · sklearn-compatible API

import xlearn as sklearn
LinearRegression · KMeans · PCA

Estimator Calls
fit · predict · transform

COMPATIBILITY · xlearn/__init__.py

__getattr__
lazy module loading

_auto_jax_accelerate_module
runtime class injection

AcceleratorRegistry
algorithm implementation map

JAX ACCELERATION · xlearn/_jax

EstimatorProxy
fit-time dispatch

Universal JAX
JIT · vmap · accelerated estimator

sklearn Fallback
original estimator
graceful fallback

DATA MANAGEMENT · boundary conversion

NumPy → JAX
dtype · shape normalization

Policy / Thresholds
dataset size · platform · config

JAX → NumPy
sklearn-compatible outputs

HARDWARE ABSTRACTION · JAX backend

CPU
jaxlib

CUDA GPU
JAX XLA

TPU
JAX XLA

Apple Metal
experimental

代理根据算法、数据规模和配置选择 JAX；不满足条件或执行失败时回退原生 sklearn。

Secret-Learn — 六层架构 · 隐私计算数据流

sklearn-compatible API → privacy mode → JAX / SecretFlow compute → secure devices

Primary flow
Fallback / feedback
Dashed = architecture zone
Box = module + responsibility

PROJECT OVERVIEW Secret-Learn exposes sklearn-compatible machine learning across federated, secret-sharing, and split-learning modes.
It combines JAX acceleration with SecretFlow devices so each workflow can choose where plaintext, shares, and activations reside.

APPLICATION · 多方隐私数据

- **Healthcare / Finance** — cross-party modeling
- **IoT / Research** — distributed learning

SKLEARN API · 统一算法入口

- **secretlearn.*** — fit · predict · transform
- **algorithm_classifier.py** — classify fit signature / epochs

PRIVACY MODES · execution placement

- **FEDERATED LEARNING** — PYU local compute; HEU secure aggregation
- **SECRET SHARING** — SPU MPC · ABY3 / CHEETAH
- **SPLIT LEARNING** — split model · encrypted activations

ACCELERATION & COMPUTE

- **JAX Acceleration** — JIT · vmap · xlearn
- **SecretFlow Runtime** — party orchestration

SECURE DEVICES

- **PYU** — party-local plaintext
- **HEU** — homomorphic aggregation
- **SPU** — secure MPC
- **TEE** — optional enclave

INFRASTRUCTURE & HARDWARE

- **SecretFlow** — distributed secure runtime
- **JAX / XLA** — CPU · GPU · TPU
- **sklearn** — API compatibility

同一 sklearn 调用由 FL / SS / SL 决定数据驻留与执行设备；JAX 加速可公开的本地计算。

YouTube-DNN-RecSys — 系统架构 · 召回数据流

Python / Spark data preparation → TFRecords → dnn.py → user & item vectors → prediction

Primary flow
Fallback / feedback
Dashed = architecture zone
Box = module + responsibility

PROJECT OVERVIEW YouTube-DNN-RecSys is a TensorFlow and Spark implementation of the candidate-generation stage of YouTube recommendations. It turns behavior histories into TFRecords, learns user and item embeddings, and produces Top-K recall candidates for serving.

DATA PROCESSING · 离线样本构建

Python Pipeline
data2tfrecords.py
read_tfrecords.py

Spark Pipeline
MakeDataOne.scala
MakeDataTwo.scala

FeatureBuilder
history + continuous features

STORAGE · 训练与模型产物

TFRecords
train / evaluation

Checkpoints
ckpt / eval / modelpath

MODEL TRAINING · src/python/models/dnn.py

Input
visit_items_index × 5
continuous_features × 16

Item Embedding
500 × 16 · average pooling

User Tower
Dense 64 → 32 → vector 16

Softmax Output
500 classes · Top-K

evaluation feedback

PREDICTION · serving paths

Python SavedModel
load_dnn_model.py

User Vector
PredictUserVector.scala

Item Table
ItemEmbeddingPredictor.scala

RECOMMENDATION OUTPUT

Candidate Recall
user vector × item embedding

Top-K Items
next-item recommendation

主链：行为历史 → TFRecords → embedding mean-pool → user vector → item table matching → Top-K。

Research CLI — 系统架构 · Agent 请求数据流

Terminal UI → core orchestration & routing → tools / MCP / research modules → model providers

Primary flow
Fallback / feedback
Dashed = architecture zone
Box = module + responsibility

PROJECT OVERVIEW Research CLI is a terminal-first AI agent workflow that connects interactive sessions to models, tools, and MCP services.
Its core router manages context and provider selection while research modules cover writing, bibliography, analysis, and submission.

CLI FRONTEND · packages/cli

- **Interactive TUI** — research.tsx · sessions
- **Commands** — mcp · skills · hooks
- **ACP** — acpSession.ts

CORE ORCHESTRATION · packages/core

- **Context & Memory** — config.ts · memory.ts
- **modelRouterService** — provider selection
- **Routing Strategy** — default · classifier · approval

TOOLS & INTEGRATIONS

- **Workspace Tools** — filesystem · shell
- **MCP** — external service adapters
- **Research Modules** — writing · bibliography · analysis
- **SDK / A2A / VS Code** — embedded clients

tool result loop

MODEL PROVIDERS · provider config

- **Gemini / Vertex** — hosted models
- **OpenAI / Anthropic** — hosted models
- **Baidu / DeepSeek / Qwen** — hosted models

RESULT STREAM

- **Tool Results** — structured observations
- **Assistant Response** — streamed to terminal

主链：prompt → packages/cli → packages/core router → provider / tools → structured result → terminal。

Questions-Gen — 系统架构 · 三阶段训练数据流

Training data → SFT → GRPO → DeepSeek distillation → validation → Hugging Face / Ollama

Primary flow
Fallback / feedback
Dashed = architecture zone
Box = module + responsibility

PROJECT OVERVIEW Questions-Gen is a Qwen3-14B system for generating and evaluating mathematical competition problems through staged training. The pipeline combines supervised fine-tuning, GRPO reward optimization, DeepSeek-R1 distillation, validation, and local deployment.

TRAINING DATA · problem mixture

- **Historical Problems** — 50% source distribution
- **Variants** — 30% conditional variation
- **Novel Problems** — 20% innovation target

Stage 1 SFT · QuestionsGenTrainer.py

- **Qwen3-14B** — supervised fine-tuning
- **Stage-1 Checkpoint** — stage1-fp-merged

Stage 2 GRPO · reward optimization

- **RewardCalculator** — 0.4 difficulty · 0.3 novelty · 0.2 rigor · 0.1 diversity
- **NoveltyConstraint** — variation / duplication guard
- **GRPO Trainer** — group-relative optimization

Stage 3 Distill · teacher supervision

quality feedback

- **DeepSeekTeacher** — DeepSeek-R1 reasoning teacher
- **Final Model** — final-merged-16bit

VALIDATION

- **ModelValidator** — single checkpoint
- **BatchValidator** — algebra · geometry · calculus
- **QualityEvaluator** — clarity · difficulty · originality

DEPLOYMENT

- **Hugging Face** — three published stages
- **Ollama** — local inference
- **questions-gen CLI** — validate · batch · quality

主链：混合题库 → SFT → GRPO reward/novelty → DeepSeek-R1 distill → validate → publish。